

SPANNER

Base de datos distribuida de Google

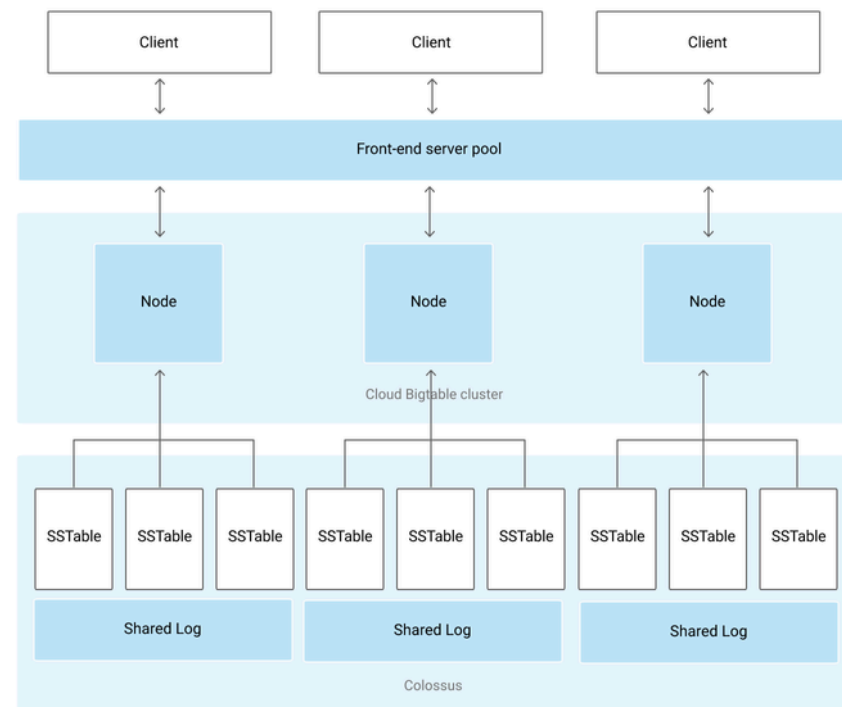
David González Villanueva
Brandon Trigueros Lara
Milton Sojo Córdoba
Jose Mena Díaz



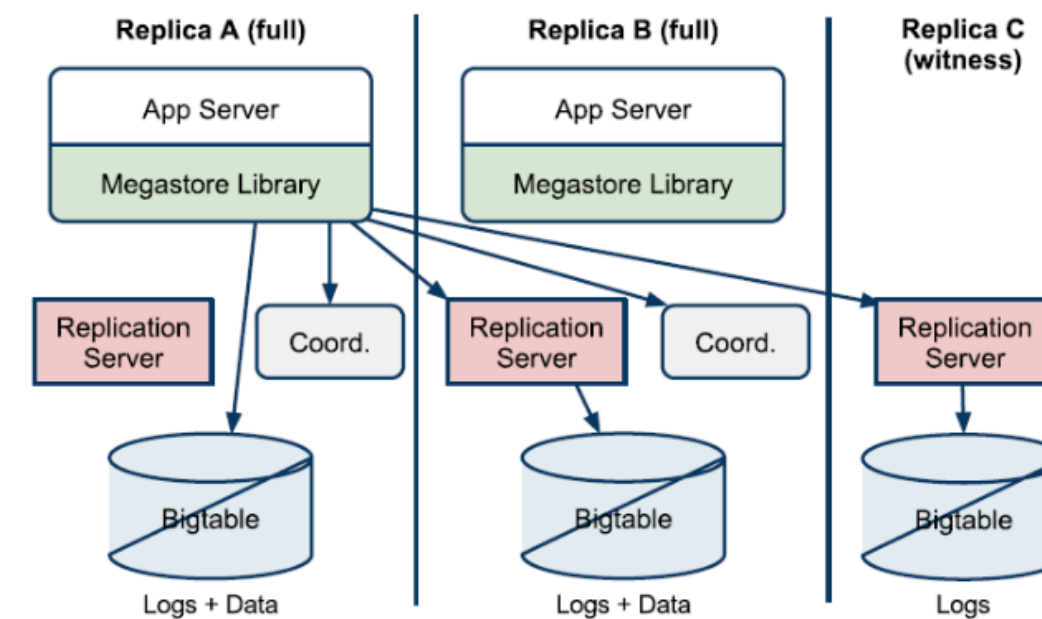
Motivación y antecedentes



Bigtable: clave-valor, replicación eventual (eventual consistency)



Megastore: semirrelacional, replicación síncrona, pero bajo rendimiento



Necesidad de un sistema que combine:

- Escalabilidad de Bigtable
- Facilidad de uso SQL y esquemas
- Alta disponibilidad global

¿Qué es Spanner?

- Sistema de base de datos **distribuido globalmente, multi-versión, replicada sincrónicamente**.
- Fue el primer sistema en soportar transacciones distribuidas con **consistencia externa** (linearizabilidad global)
- Diseñada para atender aplicaciones a escala planetaria
- Ofrece transacciones ACID y fuerte consistencia entre centros de datos

Distribuido globalmente



¿Qué significa?

Que los datos están guardados en muchos lugares del mundo al mismo tiempo.

Ejemplo:

Hay copias de los datos en diferentes continentes, así si se cae un datacenter en EE.UU., otro en Europa puede seguir trabajando.

Multi-versión



¿Qué significa?

Guarda varias versiones de un mismo dato, cada una con su propia marca de tiempo.

Ejemplo:

Si actualizo el precio de un producto de \$1000 a \$1500, Spanner guarda ambas versiones, y puedo pedir ver el precio que tenía ayer.

Replicado síncronamente



¿Qué significa?

Cada vez que se guarda algo, se guarda en varias copias al mismo tiempo, y hasta que todas lo confirmen, no se considera guardado.

Ejemplo:

Guardar un dato en Spanner es como enviar un mensaje a 3 amigos y esperar a que todos digan 'recibido' antes de continuar.

Consistencia externa



¿Qué significa?

Que las cosas pasan en el orden en que se ven.

Ejemplo:

Si alguien guarda 'Bigotón tiene \$5000', y después otra persona guarda 'Bigotón tiene \$3000', y yo leo el dato después, siempre veré \$3000. Nunca \$5000 después de \$3000.

TrueTime (servicio de reloj)

- API que devuelve intervalos de tiempo con incertidumbre
 - `TT.now()` → [earliest, latest]
 - `TT.after(t)` / `TT.before(t)`
- Relojes GPS + atómicos → incertidumbre $\varepsilon < \sim 10$ ms
- Permite el commit-wait, asegurando orden real de transacciones



Despliegue global

Único universo gestionado globalmente

- Toda la infraestructura de Spanner se trata como un solo sistema lógico, aunque distribuido en múltiples continentes.

Entornos aislados

- Cada universo incluye ambientes de prueba, desarrollo y producción, lo que facilita validación y despliegues controlados.

Escala masiva

- Opera en millones de máquinas conectadas, repartidas en cientos de datacenters.
- La gestión unificada permite que los cambios de configuración (p. ej., añadir nuevas zonas) se propaguen automáticamente sin tiempo de inactividad.

Zonas y zonemaster

División en zonas

- Un universo se particiona en varias “zonas”, que corresponden a grupos de servidores en ubicaciones geográficas o propósitos específicos.
- Cada zona actúa como un cluster lógico similar a un despliegue de Bigtable.

Zonemaster

- Es el nodo controlador de la zona: mantiene el directorio de tablets y sus réplicas.
- **Asignación de tablets:** decide en qué spanserver almacena cada fragmento de datos, equilibrando carga y requisitos de réplica
- **Agregación de métricas:** recopila estado de salud y métricas de uso de la zona para diagnóstico y escalado automático.

Spanservers y tablets

Spanservers

- Cada zona alberga cientos a miles de spanservers.
- Actúan como nodos de almacenamiento y procesamiento de lecturas/escrituras.

Almacenamiento sobre Colossus

- Cada zona alberga cientos a miles de spanservers.
- Actúan como nodos de almacenamiento y procesamiento de lecturas/escrituras.

Tablets

- Un spanserver maneja entre 100 y 1000 tablets, cada uno es un fragmento ordenado del espacio de claves.
- Un tablet define el mapeo: (clave: String, timestamp: Int64) → valor: String
- Permite almacenar versiones históricas de cada celda y habilita lecturas antiguas (snapshot reads).

Location Proxy y gestión global

Location Proxy

- Servicio por zona que mapea claves de cliente al spanserver responsable.
- Oculta la complejidad del enrutamiento y asegura baja latencia al dirigir la petición al nodo más cercano.

Universe Master

- Componente único (singleton) que ofrece una vista global de todo el universo: estado de zonas, tablets, réplicas y métricas agregadas.
- Herramienta principal para administradores y operadores.

Location Proxy y gestión global (cont.)

Placement Driver

- Servicio que mueve directorios/tablets entre zonas en segundo plano
- Balanceo de carga: ajusta dinámicamente la ubicación de fragmentos según uso y latencias.
- Gestión de réplicas: añade o quita réplicas para cumplir políticas de durabilidad y localización.

Alta disponibilidad automática

- Frente a la caída de un spanserver o zona, otros nodos detectan la pérdida de líder (lease expirada) y eligen uno nuevo, sin intervención manual.
- Esto garantiza que las operaciones continúen con mínimo impacto en el servicio.

Tablas esquematizadas con Primary Keys

¿Qué son?

- **Diagrama de filas y columnas con un esquema definido.**

Importancia

- **Facilita el particionado (sharding) y la agrupación física de datos.**

Tablas

- **Filas**
- **Columnas**
- **Valores versionados**



Versionado de celdas y Snapshot Reads

Multi-versión

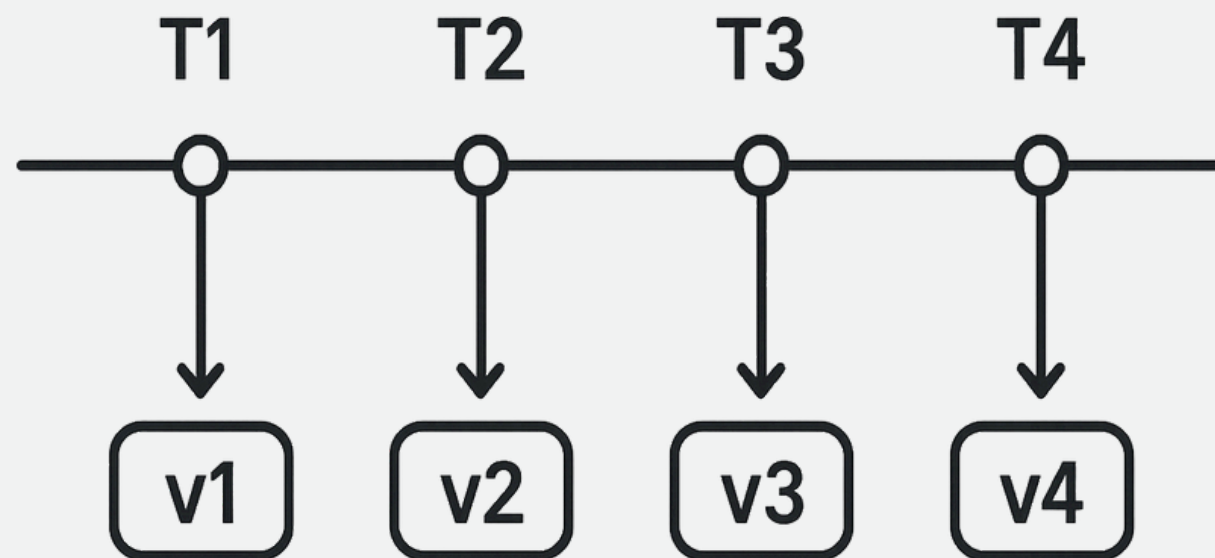
- **El sistema guarda historial; puedes leer datos en un instante pasado.**

Snapshot Reads

- **Lecturas consistentes a un timestamp concreto sin bloquear escrituras concurrentes.**

Versionado de celdas

- **Cada vez que se actualiza un valor, se crea una nueva versión con su timestamp de commit.**



Tablas Intercaladas (INTERLEAVE IN) y Directorios

INTERLEAVE IN

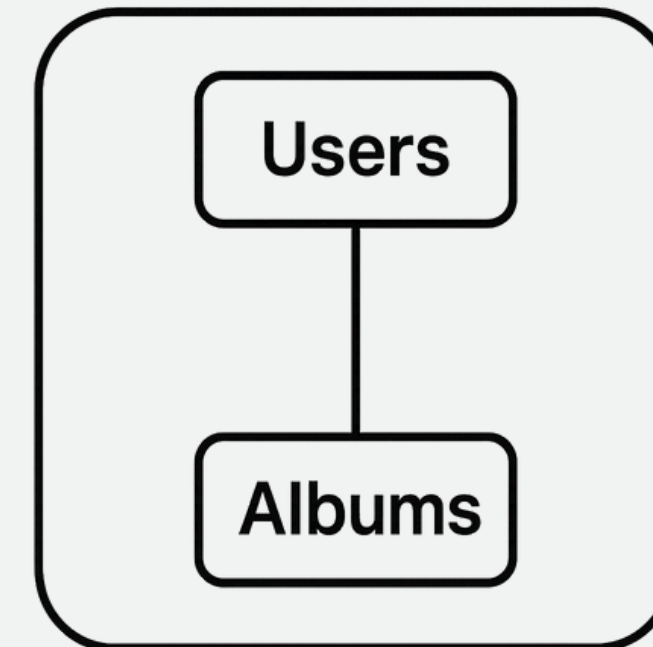
- Almacena filas de tabla hija dentro del mismo fragmento que la tabla padre.

Ventaja

- Consultas que unen datos relacionados leen un único fragmento físico.

Directorios

- Un "directorio" es una raíz con sus tablas descendientes; se mueven juntos.



TT.now()

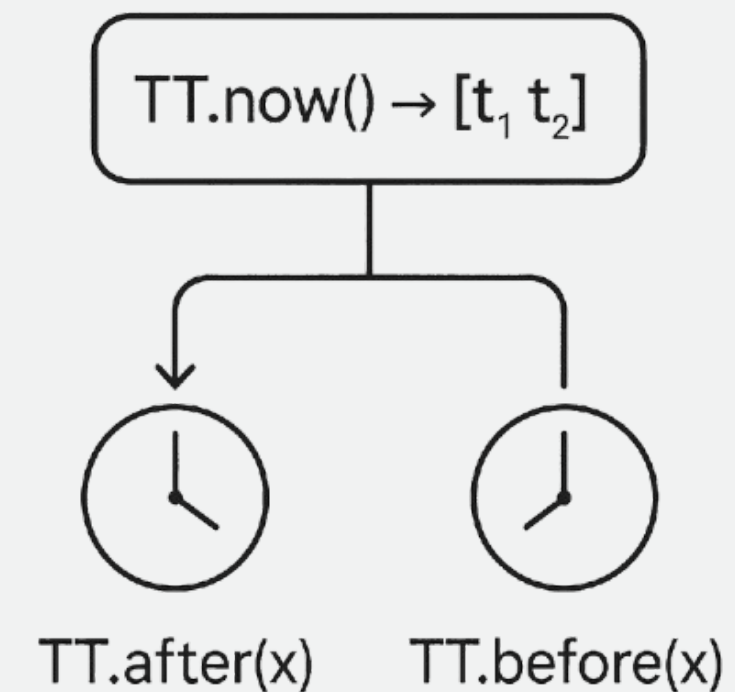
- Devuelve un intervalo “[earliest, latest]” que contiene el tiempo real.

TT.after(t)

- True si el instante t ya pasó con certeza.

TT.before(t)

- True si el instante t aún no ha llegado con certeza.

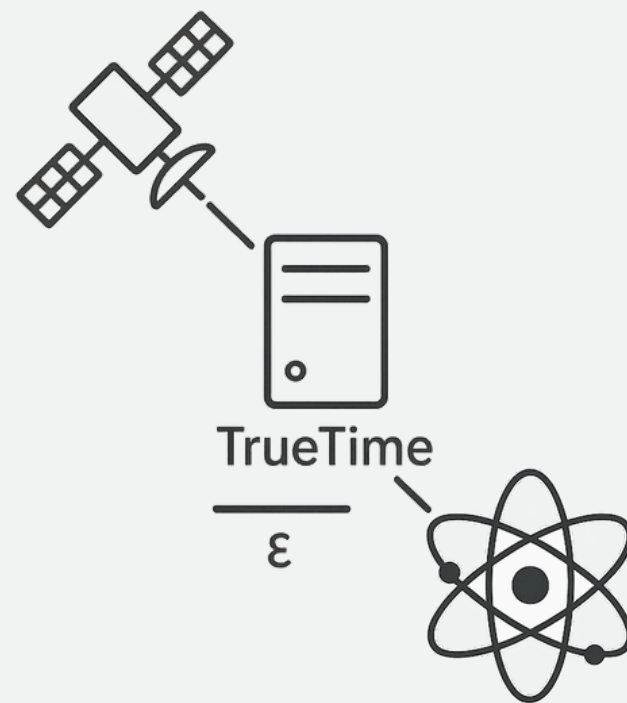


ϵ pequeño



Fuentes de tiempo

- **Satélites GPS y relojes atómicos de alta precisión.**



Incumplimiento (ϵ)

- **Margen de error garantizado (5ms).**

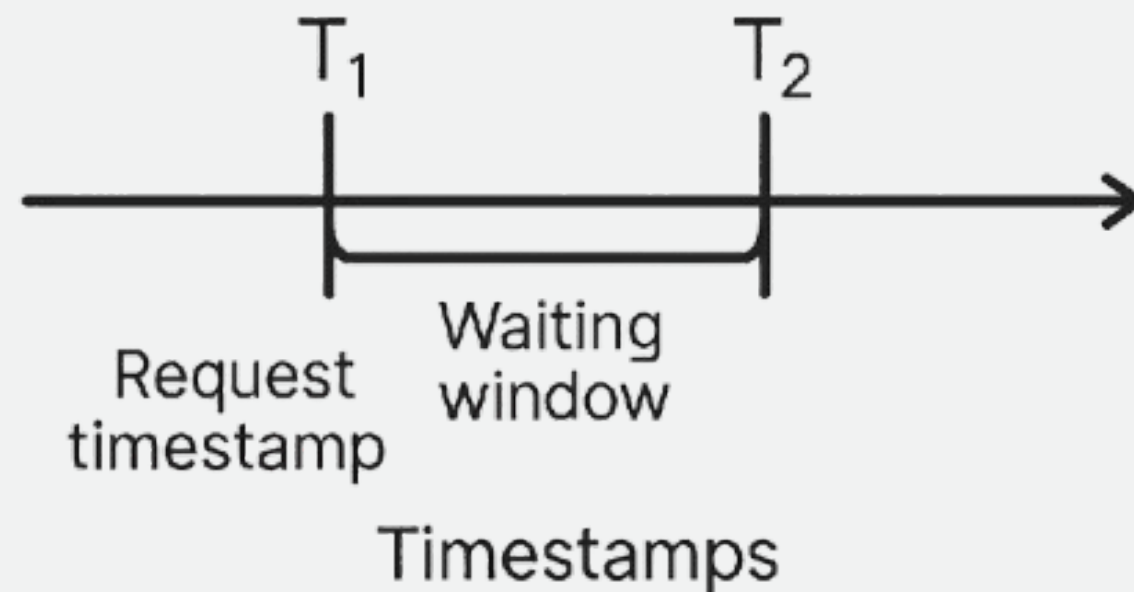
Resultado

- **Los intervalos de TrueTime son muy estrechos.**

Uso de TrueTime para Commit Wait y Consistencia Externa

Pasos

- El coordinador pide $\text{timestamp} \geq \text{TT.now().latest}$.
- Tras preparar el commit, espera a que $\text{TT.after(timestamp)}$ sea verdadero.
- Publica el resultado sólo después de ese punto.



Objetivo

- Garantizar que $\text{ts}(T_1) < \text{ts}(T_2)$ si T_1 finaliza antes de que T_2 comience.

Transacciones y Consenso Paxos



- **Transacciones ACID en Spanner**
- **Control de concurrencia híbrido**
- **Introducción al consenso Paxos**



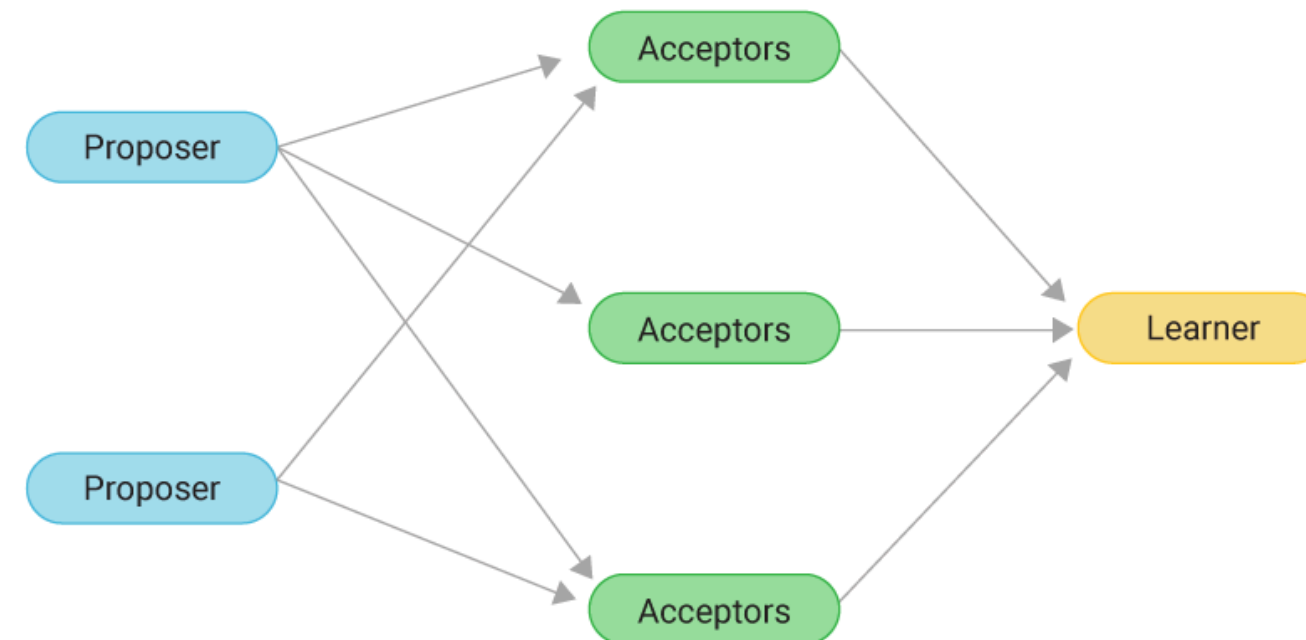
¿Qué es Paxos?



- **Algoritmo de consenso tolerante a fallos**

- **Permite que réplicas acuerden un valor único**

- **Basado en fases de proposición y aceptación**



Roles en Paxos

Proponente:

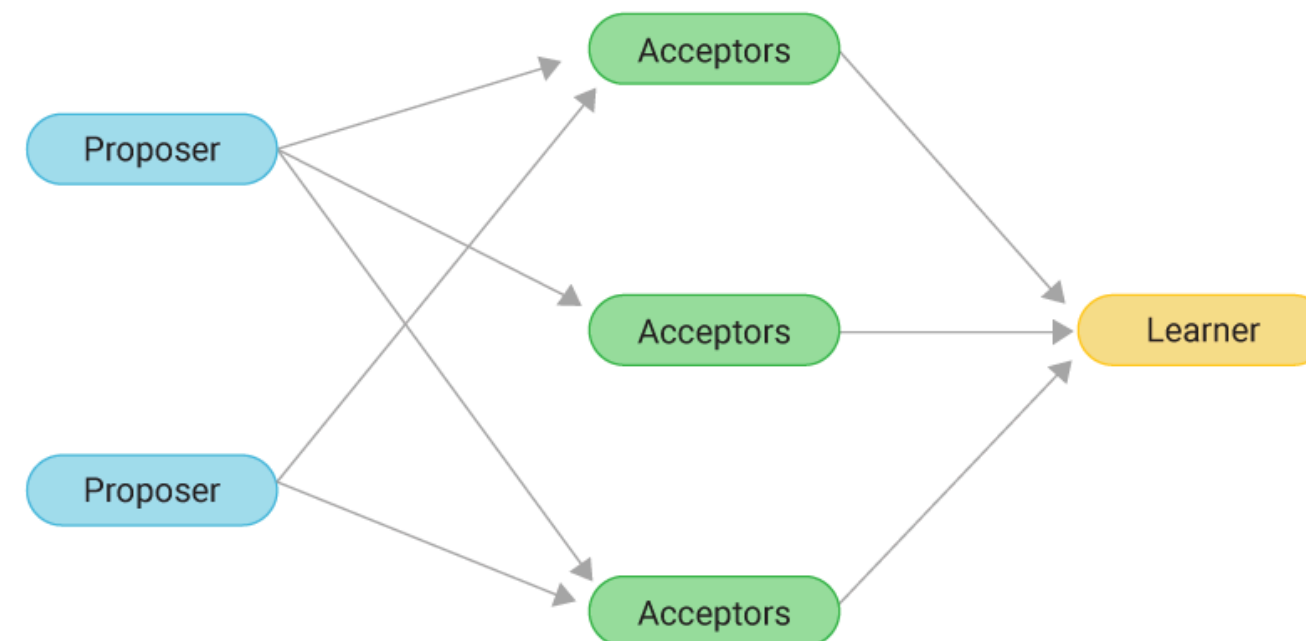
- Propone un valor (líder)

Aceptador:

- Vota por propuestas

Cliente/Aprendiz:

- Solicita cambios y observa resultados



Protocolo de Fases



Fase 1 (Prepare):

El líder envía número de propuesta

Fase 2 (Accept):

Aceptadores votan y responden

Commit:

Líder difunde valor aceptado

Implementación en Spanner



Multi-Paxos con líder
estable (lease 10s)

Cada tablet tiene
grupo de 3–5 réplicas

Escrituras requieren
mayoría de votos

Garantías y Failover



Consistencia síncrona
entre réplicas

Tolerancia a falla de
hasta f nodos

Nueva elección de Líder
al expirar lease o fallo

Replicación y disponibilidad

Sinc. multinivel

Spanner apoya replicación dentro de la capa de almacenamiento y replicación a nivel de base de datos. A nivel superior se requieren votos de mayoría en Paxos para cada escritura.

Tolerancia a fallos de zona

Si una zona completa se desconecta, mientras quede una mayoría de réplicas activas Spanner puede seguir operando. Google ha reportado que en F1 los fallos de clúster han sido prácticamente invisibles: Spanner hace failover automático y, en los pocos casos de fallo masivo, sólo fue necesario reubicar líderes.



Reubicación automática

Un componente de Spanner redistribuye datos en background para equilibrar carga y cumplir requisitos de réplica.

Cuórum y lecturas fuertes

Cualquier réplica con la última información puede servir lecturas fuertes (desde un timestamp reciente garantizado).

Caso de uso del sistema: F1 de Google

F1 se trasladó a Spanner en 2011 y es el primer cliente de producción mencionado en el paper. En F1 Spanner gestiona datos globales de clientes y anuncios con transacciones multi-usuario.

Motivación: F1 usaba MySQL y Bigtable, pero eso impedía transacciones globales consistentes y queries cruzados de datos. F1 requería replicación síncrona para tolerancia a fallos, transacciones de baja latencia que abarcaran cualquier dato, y consistencia fuerte. Spanner satisfizo esto: elimina el re-sharding manual, da replicación automática con failover sincrónico y ofrece transacciones ACID por toda la BD.



Caso de uso del sistema: F1 de Google

F1 se trasladó a Spanner en 2011 y es el primer cliente de producción mencionado en el paper. En F1 Spanner gestiona datos globales de clientes y anuncios con transacciones multi-usuario.

Despliegue: F1 usa 5 réplicas. Esto le permite sobrevivir a desastres regionales sin interrumpir el servicio. Gracias a esto y al conmutador automático de Spanner, los ingenieros de F1 reportaron prácticamente cero impacto visible cuando hubo fallos de clúster; en el peor caso solo ajustaron el esquema para preferir líderes cerca de donde migraron sus frontends.



Caso de uso del sistema: F1 de Google

F1 se trasladó a Spanner en 2011 y es el primer cliente de producción mencionado en el paper. En F1 Spanner gestiona datos globales de clientes y anuncios con transacciones multi-usuario.

Funcionalidad: F1 implementó índices secundarios consistentes usando transacciones de Spanner. También aprovecha la semántica de timestamp: mantiene un log de cambios internos y snapshots periódicos consultando Spanner a ciertos timestamps. De hecho, en cada transacción de F1 se escribe parte de este log histórico, y luego F1 lee snapshots para reconstruir sus estructuras en memoria.



Conclusiones

Spanner aporta consistencia fuerte y replicación mundial a bases de datos relacionales, sobre una infraestructura masiva. Combina lo mejor de dos mundos: la facilidad de uso de SQL/ACID y la robustez de los sistemas distribuidos. Sus innovaciones (especialmente TrueTime) han influido en la comunidad: ya no debemos confiar en relojes imprecisos cuando diseñamos sistemas distribuidos.



GARCIAS